codehive

Python: Reading Files

Methods for Extracting Data & Content

1. The File Object

When you open a file in Python, the `open()` function returns a **File Object**.

> This object is your interface to the data. It contains methods like `read()`, `readline()`, and iteration support to pull data from the disk into your program's memory.

2. Reading the Entire File

By default, the `read()` method extracts the entire contents of the file as a single string.

```
f = open("demofile.txt", "r")
content = f.read()
print(content)
f.close()
```

This is convenient for small files, but consuming very large files all at once can deplete your system's RAM.

3. The 'with' Statement

Manual closing via `f.close()` is prone to error. The `with` statement (Context Manager) is the **industry standard**.

```
with open("demofile.txt") as f:
    data = f.read()
# File is automatically closed here
```

Even if an error occurs inside the block, Python ensures the file is released properly.

4. Partial Reading

You can control exactly how much data you pull by passing an integer to `read(n)`, which returns the first n characters.

```
with open("demofile.txt") as f:
    print(f.read(5)) # Output: Hello
```

Subsequent calls to `read()` will continue from where the previous one left off.

5. Reading Line by Line

The `readline()` method reads a file until it hits a newline character (\n).

```
with open("demofile.txt") as f:
    line1 = f.readline()
    line2 = f.readline()
    print(line1)
    print(line2)
```

This is useful for header-based files like CSVs where you only want the first row initially.

6. Iterating Over Files

For large datasets, the most memory-efficient way is to loop through the file object directly. This loads only one line into memory at a time.

```
with open("demofile.txt") as f:
    for line in f:
        print(line.strip())
```

Using `strip()` removes the extra newline character added by the file and the `print()` function.

7. Buffered Reading

Python uses buffering to make file access faster. However, this means changes might not be saved to disk until the file is closed.

Important: Always close or use `with`. If your program crashes before a file is closed, you risk data corruption or loss.

8. Relative vs Absolute Paths

Relative: `open("data.txt")` - Only works if the file is in the script's directory.

Absolute: `open("C:\\Users\\Admin\\data.txt")` - Works from anywhere on the system.

> On Windows, remember to use double backslashes (\\) to avoid escape character errors.

9. Practical Use-Case

Pattern Checklist

1. Open the context manager.
2. Choose the extraction method (read, readline, or loop).
3. Process the data (clean strings, parse numbers).
4. Exit block (automatic cleanup).

10. Core Summary

- ✓ `read()` pulls everything.
- ✓ `read(5)` pulls a specific count.
- ✓ `readline()` pulls one line at a time.
- ✓ `for x in f` is the most efficient loop.
- ✓ **Always** prioritize the `with` keyword.

codehive

Data Extraction Series v1.2